

Risk Propagation Audit

Repository: S:\FYP Audit date: 2026-09-08 Scope: DataSense ingestion, feature extraction, model evidence, gateway admission, device ABM state, topology propagation, SREP, backend projection, and frontend display.

This audit is based on executable implementation, contracts, configuration, repository data metadata, and automated tests. Comments and methodology documents are used as supporting evidence, not as substitutes for tracing executable code. The raw PCAP and NDJSON corpus files are not present in this checkout, so raw-field claims were checked against the executable parsers and the repository's prior raw audit rather than by rescanning every original capture. The inventory CSV and saved smoke-model split manifest are present and were inspected directly.

1. Executive conclusion

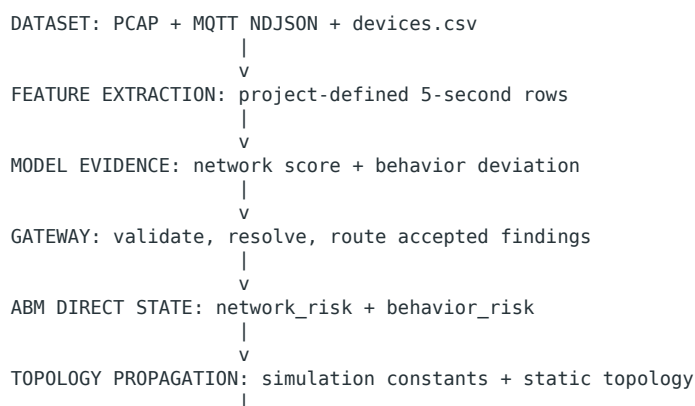
Answer to the main question: risk propagation is not present as a field or coefficient in the original DataSense records, and neither ML model infers a propagated-risk value. The network model emits a direct class-1 score and the behavior profiler emits a direct anomaly/deviation score. After gateway admission, DeviceABM.propagate() synthesizes propagated risk over a project-constructed topology using simulation-defined constants. The same ABM then defines systemic risk as the maximum of present direct evidence and propagated risk (simulation/abm.py:126-179, config.py:24-57).

```
| Does original DataSense contain this? | Verdict | Actual origin and evidence |
| network_risk | NO | It is assigned from NetworkFinding.attack_probability by DeviceABM.apply_network_evidence()
(simulation/abm.py:103-111). The finding is produced by the Random Forest (pipeline/network_detector.py:117-176). |
| behavior_risk | NO | It is assigned from BehaviorFinding.deviation_score by DeviceABM.apply_behavior_evidence()
(simulation/abm.py:113-123). The profiler creates the score (pipeline/behavior_profiler.py:281-344). |
| propagated_risk | NO | It is recomputed by DeviceABM.propagate() (simulation/abm.py:126-169). |
| systemic_risk | NO | It is max-fused by DeviceABM.propagate() after propagation (simulation/abm.py:168-179). |
| Attack-propagation paths | NO as risk paths; PARTIALLY as structural evidence | DataSense supplies device identities and
observed packets. The propagation graph is a hard-coded nx.DiGraph whose nodes come from inventory and whose edge rules are
labelled documented or strongly inferred (simulation/topology.py:41-84). It is not learned from packet communication. |
| Risk-propagation weights | NO | propagation_weight=0.5 is project configuration explicitly labelled simulation-defined
(config.py:24-27,39-43). |
| Hop-decay values | NO | hop_decay=0.5 and max_hops=3 are project configuration (config.py:39-43). |
```

The provenance layers are therefore:

1. Raw dataset evidence: packet records, packet headers, MQTT telemetry, timestamps, and inventory identity (datasets/datasense/pcap_reader.py:41-47, datasets/datasense/frame_decoder.py:55-83, datasets/datasense/ndjson_reader.py:25-40, data/raw/datasense/docs/site/devices.csv:1-46).
2. Extracted features: per-device/per-window network and behavior aggregates (datasets/datasense/network_features.py:48-118,426-521, datasets/datasense/behavior_features.py:43-98,397-497).
3. Model-produced evidence: network class-1 score and behavioral deviation score (pipeline/network_detector.py:117-176, pipeline/behavior_profiler.py:281-344).
4. Simulation-defined state: propagated risk, systemic fusion, compromise rule, role criticality, and defended blast radius (simulation/abm.py:103-189, config.py:39-57).
5. Topology-derived state: propagation traverses the static project topology, not the dynamic communication graph (simulation/topology.py:41-84, simulation/communication_graph.py:1-16).
6. Frontend/SREP projections: backend and browser copy and format ABM values; they do not recalculate them (backend/app/adapters/stage2_replay_adapter.py:195-228, frontend/src/components/devices/DeviceStateTable.tsx:4-7,69-75,110-121, srep/device_srep.py:52-108).

Diagram 1: provenance layers

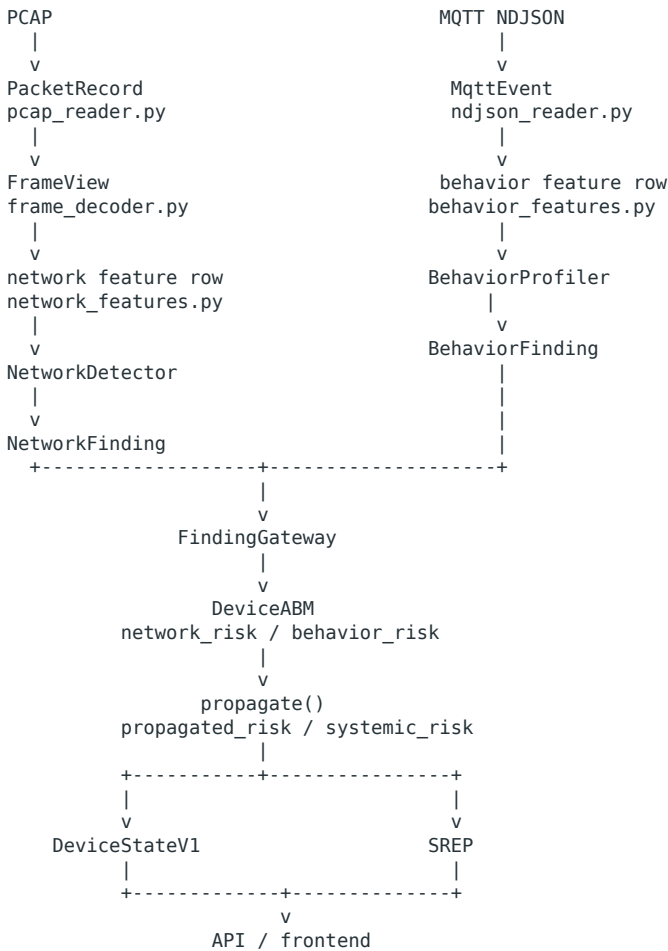


```

      v
SYSTEMIC STATE: max(direct risk, propagated risk)
      |
      v
BACKEND CONTRACTS --> SREP / REST / WebSocket --> FRONTEND

```

2. End-to-end data lineage



Transition	Input	Output	Owner	Classification
PCAP read	capture bytes	PacketRecord(ts_ns, caplen, wirelen, data)	iter_packets() and PcapPacketStream,	
		datasets/datasense/pcap_reader.py:276-324		DATASET-DERIVED container record
Frame decode	PacketRecord.data	FrameView	decode_frame(), datasets/datasense/frame_decoder.py:149-273	DATASET-DERIVED header decode
Network extraction	timestamp, FrameView, lengths	flat network row	NetworkWindowManager.add_packet()/finalize(), datasets/datasense/network_features.py:258-322,426-521	EXTRACTED / PROJECT-DERIVED
MQTT read	NDJSON object	MqttEvent	parse_telemetry_line()/iter_mqtt_events(), datasets/datasense/ndjson_reader.py:53-139	
		DATASET-DERIVED compact event		
Behavior extraction	MqttEvent	flat behavior row	BehaviorWindowManager.add_event()/finalize(), datasets/datasense/behavior_features.py:237-276,397-497	EXTRACTED / PROJECT-DERIVED
Network inference	observed network row	NetworkFinding	NetworkDetector.findings_from_records(), pipeline/network_detector.py:142-176	MODEL-DERIVED
Behavior inference	behavior row	BehaviorFinding or no finding	BehaviorProfiler.predict_record(), pipeline/behavior_profiler.py:281-344	MODEL/RULE-DERIVED
Admission	finding	accepted/rejected; accepted finding mutates one direct channel	FindingGateway.submit(), agents/finding_gateway.py:62-123	VALIDATION/ROUTING
Direct state	attack score	network_risk; deviation -> behavior_risk	DeviceABM.apply_*_evidence(), simulation/abm.py:103-123	MODEL-DERIVED VALUE IN SIMULATION STATE
Propagation	direct state + static topology + constants	propagated_risk	DeviceABM.propagate(), simulation/abm.py:126-169	SIMULATION-DEFINED
Fusion	direct and propagated	systemic_risk	DeviceABM.propagate(), simulation/abm.py:168-179	SIMULATION-DEFINED
Projection	ABM fields	DeviceStateV1	device_state_contracts(), backend/app/adapters/stage2_replay_adapter.py:195-228	PASS-THROUGH
Display	DeviceStateV1	text/bar	DeviceStateTable, frontend/src/components/devices/DeviceStateTable.tsx:4-7,69-75,110-121	PRESENTATION ONLY

datasets/datasense/extraction.py is the raw-source orchestration point. A single PCAP pass invokes both network and communication managers (datasets/datasense/extraction.py:126-245), while MQTT records are sorted by window before behavior accumulation (datasets/datasense/extraction.py:312-482). Stored and direct-raw modes expose the same row dictionaries (datasets/datasense/feature_store.py:322-394).

3. Network risk

Meaning and calculation

`network_risk` is the latest gateway-accepted Random Forest class-1 score for a device. It is not a dataset column and is not propagated by the network model.

1. The extractor defines exactly 57 numeric/statistical model inputs, covering packet counts, timing, address/port/protocol diversity, flags, fragmentation, sizes, TTL, TCP window, and MSS (datasets/datasense/network_features.py:48-106). Exact identities and graph metadata are separate (datasets/datasense/network_features.py:108-118).
2. `_rows_to_matrix()` selects exactly those fields (pipeline/network_detector.py:43-48).
3. Training uses median imputation, standard scaling, then `RandomForestClassifier` (pipeline/network_detector.py:65-78). The default forest has 200 trees, balanced class weights, minimum leaf size 2, and random seed 42 (pipeline/network_detector.py:51-60).
4. Runtime calls the fitted pipeline's raw `predict_proba()` and selects the probability-like score for class 1 (pipeline/network_detector.py:117-139).
5. The threshold is 0.5: `score >= 0.5` gives class attack; otherwise benign. Confidence is the selected class's score, `p` or `1-p` (pipeline/network_detector.py:132-139).
6. The score becomes `NetworkFinding.attack_probability` (pipeline/network_detector.py:142-176), and after gateway admission it is copied without averaging to `state.network_risk` (agents/finding_gateway.py:96-109, simulation/abm.py:103-111).

Is it calibrated?

It is a classifier probability-like score, not a demonstrated calibrated probability. The pipeline has no `CalibratedClassifierCV`, Platt scaling, isotonic regression, or other calibration stage (pipeline/network_detector.py:68-76). The saved smoke manifest reports classification metrics but no calibration metric; it also reports only 481 total positives, zero validation recall, and no positives in the test split (models/saved_models/split_manifest_smoke.json:27-52). It should therefore not be described as a validated probability that the device is attacked.

Training labels are target-aware, evaluation-only labels: observed explicit targets and observed protected assets in whole-network attacks are positive; genuine benign capture windows are negative; unobserved and non-target attack context rows are excluded (pipeline/ground_truth.py:1-22,67-92). Labels are not runtime inputs.

Missingness and state effects

- A dense network row with no evidence has `network_observed=False` and all model features `None` (datasets/datasense/network_features.py:524-545).
- The detector rejects unobserved rows (pipeline/network_detector.py:117-128), and replay filters them before inference (simulation/replay.py:287-295).
- Initially, `network_risk=None` means no accepted network finding is stored (simulation/abm.py:33-49). After a finding has been accepted, later unobserved windows do not clear it. Therefore a non-null value means latest accepted evidence at some earlier or current replay window, not necessarily evidence from the currently displayed window (simulation/abm.py:103-111).
- The detector never modifies ABM state directly. It returns immutable finding objects (pipeline/network_detector.py:142-209); the gateway is the mutating boundary (agents/finding_gateway.py:104-109).
- `compromised=True` is set only when an accepted network finding has `attack_probability >= 0.5` and the node is protected (simulation/abm.py:103-110). Section 13 distinguishes this from systemic risk.

4. Behavior risk

Meaning

`behavior_risk` is the latest accepted per-device behavioral deviation/anomaly score, not an attack probability and not a supervised attack classifier. The profiler is trained on each supported sensor's chronological benign baseline (pipeline/behavior_profiler.py:1-18,104-158). The resulting `BehaviorFinding.deviation_score` is copied directly into ABM state (pipeline/behavior_profiler.py:334-344, simulation/abm.py:113-123).

Extracted inputs

Raw MQTT fields are parsed into `MqttEvent` (`datasets/datasense/ndjson_reader.py:25-40,53-90`). Behavior extraction creates 19 common features, 14 continuous/degenerate features, and 3 sparse features (`datasets/datasense/behavior_features.py:43-98`). Nonapplicable fields are null rather than zero (`datasets/datasense/behavior_features.py:486-497`).

Profile categories are project assignments, not an original CSV field:

- Continuous: nine named high-rate sensors.
- Sparse: motion, RFID, flame, and proximity-collision sensors.
- Degenerate: the water sensor.
- Unsupported: non-sensors and unrecognized sensor names.

The exact mapping is in `datasets/datasense/devices.py:19-50,113-123`.

Continuous profile

Continuous profiles use a 16-field list dominated by cadence, burst, topic/type, QoS, retained, duplicate, and message-ID counts (`pipeline/behavior_profiler.py:45-62`). Missing feature values are converted to numeric zero by `_vector()` (`pipeline/behavior_profiler.py:82-87`). Absolute value levels are excluded unless an explicit ablation flag adds them (`pipeline/behavior_profiler.py:64-69,161-168`).

An `IsolationForest(n_estimators=100, contamination=0.05, random_state=42)` is fit per sensor (`pipeline/behavior_profiler.py:161-168`). Its runtime score is:

```
raw = -IsolationForest.decision_function(x)
behavior_score = clamp((raw + 0.5) / 1.5, 0, 1)
```

The transformation is at `pipeline/behavior_profiler.py:389-407`; the same affine mapping is used for calibration scores at `pipeline/behavior_profiler.py:170-180`. It is an engineering transformation, not probability calibration. A threshold τ is selected at approximately the 99th percentile of a later calibration block (`pipeline/behavior_profiler.py:177-180`). Every observed row still emits a finding; τ changes explanation/confidence rather than suppressing findings (`pipeline/behavior_profiler.py:315-344,404-407`).

Degenerate profile

The water sensor uses the continuous model plus guards:

- an expected constant stream becoming variable raises score to at least 0.85;
- value drift over 5 percent raises score to at least 0.8.

See `pipeline/behavior_profiler.py:182-191,393-403`. A caveat is that extraction forces `constant_value_stream=True` for the degenerate profile (`datasets/datasense/behavior_features.py:489-493`), making the first guard effectively unreachable for ordinary extracted water rows. The value-drift guard can still fire.

Sparse profile

Sparse profiles have no fitted ML model (`model=None`). Training derives active frequency, mean active message count, and:

```
absence_tau_windows = max(3, int(percentile(active_window_gaps, 90)) + 1)
```

(`pipeline/behavior_profiler.py:219-277`). For an observed sparse row, the additive rule score is:

```
| Rule | Increment |
| zero messages | 0.40 |
| messages > max(2, 3 * mean_active_msgs) | 0.35 |
| time since previous event | min(0.25, gap_seconds / 600) |
| more than two binary state flips | 0.15 |
```

The total is capped at 1 (`pipeline/behavior_profiler.py:409-432`). Its τ is approximately the 99th percentile of calibration rule scores (`pipeline/behavior_profiler.py:258-277`).

Absence evidence

An unobserved sparse row can produce inferred behavior evidence only when:

1. replay says telemetry context is active;
2. the profiler has seen a previous active window for that sensor;
3. the current gap exceeds `absence_tau_windows`.

Then:

```
over = gap_windows - absence_tau_windows
deviation_score = min(1, 0.5 + 0.1 * over)
confidence = min(1, 0.5 + 0.05 * min(over, 10))
```

(`pipeline/behavior_profiler.py:295-313,346-387`). Replay defines context as any behavior row in the window having

behavior_observed=True (simulation/replay.py:240-248). Complete modality absence produces no finding.

None versus 0.0

State	Meaning
behavior_risk=None	No accepted behavioral evidence is stored, or behavior is unsupported. It is unknown/unavailable, not safe.
behavior_risk=0.0	The profiler emitted and the gateway accepted a numeric deviation score of exactly zero. It is observed/model-produced evidence.

Unsupported or ordinary unobserved rows return no finding (pipeline/behavior_profiler.py:290-313). DeviceABM initializes missing behavior as None (simulation/abm.py:42-49) and rejects behavioral evidence for an unsupported state (simulation/abm.py:113-119). The backend contract preserves null (backend/app/contracts/device_state_v1.py:18-32), and the frontend renders unsupported null as N/A / Unsupported but numeric zero as 0.000 (frontend/src/components/devices/DeviceStateTable.tsx:4-7,71-75).

Like network risk, behavior risk is latched: no-observation windows do not clear a previous accepted value. Also, accepted sparse absence sets ABM behavior_observed=True, so that ABM flag means behavioral evidence was accepted, not necessarily that an MQTT message was physically observed in that window (simulation/abm.py:113-123).

5. Direct risk fusion

Direct risk is not stored as a DeviceState field. It is calculated locally by DeviceABM.propagate() and separately reconstructed for a workflow recommendation (simulation/abm.py:135-145,170-179, agent_workflow/risk_analyst.py:66-77).

For final fusion, let the set of present direct values be:

$$E_i = \{N_i \text{ if } N_i \text{ is not None}\} \cup \{B_i \text{ if } B_i \text{ is not None}\}$$

Then:

$$D_i = \max(E_i), \text{ if } E_i \text{ is non-empty}$$

$$D_i = \text{None}, \text{ otherwise}$$

This follows candidates, present, and max(present) at simulation/abm.py:170-175. Missing behavior is not averaged into network risk as zero. At source selection only, None and 0.0 both mean "not a positive source" because the code uses network_risk or 0.0 and an explicit behavior fallback (simulation/abm.py:135-144). Final missingness remains distinguishable.

network risk	behavior risk	direct result
0.8	0.2	0.8
0.2	0.8	0.8
0.8	None	0.8
None	0.8	0.8
None	None	None

Diagram 3: state composition

```

network_risk  -----+
                    +--> max of present values --> direct_risk -----+
behavior_risk  -----+
                    |
                    +--> max --> systemic_risk
propagated_risk -----+

```

6. Topology construction

Graph source and type

build_topology() creates a frozen networkx.DiGraph (simulation/topology.py:41-84). Nodes are copied from DeviceInventory, whose production loader reads device_name, MAC, IP, role, type, and main topic from devices.csv (datasets/datasense/devices.py:61-99). The inventory file contains 45 nodes, including 14 sensors, six attackers, and one cloud node (data/raw/datasense/docs/site/devices.csv:1-46).

Edges are not learned from PCAP communication and are not dynamically changed by the communication graph. They are explicit source-code rules:

- sensors and cameras -> AP;
- AP <-> switch;
- switch <-> MQTT broker;
- switch <-> edge1;
- router -> switch;
- attacker0 -> attacker1 through attacker5;
- router -> cloud;

- sensors -> MQTT broker;
- plugs -> AP.

See simulation/topology.py:22-38,56-83. Edges contain provenance and relation, but no numeric propagation weight. "Undirected" relations are encoded as two directed edges (simulation/topology.py:56-59). The graph is frozen before return (simulation/topology.py:84); this prevents structural add/remove operations, although NetworkX freezing does not make attribute dictionaries immutable.

The edge set is therefore best classified as project-constructed and partially dataset/testbed-grounded: nodes and node metadata come from inventory; some edge rules are labelled DOCUMENTED, while MQTT and plug associations are labelled STRONGLY_INFERRED (simulation/topology.py:19-20,61-82). The raw communication graph is separate and explicitly not used for risk propagation (simulation/communication_graph.py:1-16).

Directional consequence of nx.all_neighbors

Although the graph is directed, propagation calls:

```
for neighbor in nx.all_neighbors(self.topology, node):
```

at simulation/abm.py:154. For a directed NetworkX graph, all_neighbors enumerates predecessors and successors. This audit verified that behavior against the installed NetworkX 3.6.1: for the sole edge A -> B, list(nx.all_neighbors(g, "B")) == ["A"] even though B has no successor. requirements.txt:5 does not pin a NetworkX version.

Consequently propagation treats the directed graph as effectively bidirectional adjacency. For example, a sensor -> ap arc permits propagation from AP back to the sensor, and router -> switch permits switch risk to travel back to the router. The API still projects these topology edges as directed (backend/app/adapters/stage2_replay_adapter.py:258-267), so displayed direction does not describe the constraint used by propagation.

Node classes and lifetime

- is_protected_asset is true when role is neither attacker nor cloud.
- is_attacker is true only for role attacker.
- attacker and cloud nodes exist in the graph and ABM state, but propagation filters them as described in section 11.
- topology structure is fixed for a replay; communication observations update a separate dynamic CommunicationGraph (simulation/communication_graph.py:29-38, 64-80).

These flags are set in both topology node metadata and ABM state (simulation/topology.py:43-54, simulation/abm.py:72-91).

7. Propagation algorithm

The exact implementation is DeviceABM.propagate() (simulation/abm.py:126-179). At algorithmic level:

1. Read w=propagation_weight, d=hop_decay, and H=max_hops from ABM parameters (simulation/abm.py:131-133). Defaults are w=0.5, d=0.5, H=3 (config.py:39-43).
2. For each protected node, compute source direct value as the maximum of network risk and behavior risk, treating absence as no positive source. Keep only D_s > 0 (simulation/abm.py:135-145).
3. For each source independently, initialize visited={source} and a FIFO list with (source, D_s, 0) (simulation/abm.py:146-150).
4. Pop a node. If hops >= H, do not expand it (simulation/abm.py:150-153).
5. Traverse nx.all_neighbors, therefore predecessors and successors (simulation/abm.py:154).
6. Skip already visited nodes and non-protected nodes (simulation/abm.py:155-159).
7. Calculate:

text pushed = current_risk * w * d^(hops + 1)

(simulation/abm.py:160). Values <= 1e-9 are dropped (simulation/abm.py:161-162).

8. Mark the neighbor visited, retain the maximum candidate from all sources, and enqueue it with hop count incremented (simulation/abm.py:163-166).

9. Replace every node's previous propagated risk with the newly calculated value, defaulting to 0 (simulation/abm.py:168-169).

10. Max-fuse direct and propagated values into systemic risk (simulation/abm.py:170-179).

Exact multi-hop formula

The local formula is not simply applied once to the original source value. The already-decayed value is reused at each next edge. For a source direct risk D_s and a node first reached after k traversed edges:

$$R_s(k) = D_s * w^k * d^{(1 + 2 + \dots + k)} \\ = D_s * w^k * d^{(k(k+1)/2)}$$

With defaults $w=d=0.5$:

```
| Distance from source | Value arriving |
| source | no self-propagated value from its own traversal |
| first neighbor |  $D_s * 0.5 * 0.5^1 = D_s * 0.25$  |
| second-hop neighbor |  $D_s * 0.5^2 * 0.5^3 = D_s * 0.03125$  |
| third-hop neighbor |  $D_s * 0.5^3 * 0.5^6 = D_s * 0.001953125$  |
```

The hop cap is hard: items at hop count 3 are not expanded under defaults. A node at three edges can receive risk, but no fourth edge is traversed (simulation/abm.py:149-166). These parameters are neither learned nor empirically calibrated by code. They are simulation constants, programmatically overrideable by passing `srep_params` to DeviceABM (simulation/abm.py:58-70). Production creates the ABM without an override (backend/app/adapters/stage2_replay_adapter.py:84-87).

8. Worked numerical propagation example

Consider a custom chain of protected sensor-role nodes. Because `all_neighbors` uses both directions, the arrow direction does not prevent reverse adjacency:

Diagram 2: one source

```
Sensor-A ---- Gateway-B ---- Controller-C ---- Device-D
direct 0.8      none          none          none
  |              ^              ^              ^
  +-- hop 1: 0.200000
      +----- hop 2: 0.025000
          +----- hop 3: 0.0015625
```

Calculation:

```
B = 0.8 * 0.5 * 0.5^1 = 0.200000
C = 0.2 * 0.5 * 0.5^2 = 0.025000
D = 0.025 * 0.5 * 0.5^3 = 0.0015625
```

Node	network risk	behavior risk	direct risk	propagated risk	systemic risk
A	0.8	None	0.8	0	0.8
B	None	None	None	0.2	0.2
C	None	None	None	0.025	0.025
D	None	None	None	0.0015625	0.0015625

Now give B direct network risk 0.6. B becomes an independent source and sends 0.15 to each adjacent node. Its two-hop contribution to D is 0.01875, which exceeds A's three-hop 0.0015625 contribution.

Node	network risk	behavior risk	direct risk	propagated risk	systemic risk
A	0.8	None	0.8	0.15 from B	0.8
B	0.6	None	0.6	0.20 from A	0.6
C	None	None	None	$\max(0.025 \text{ from A}, 0.15 \text{ from B}) = 0.15$	0.15
D	None	None	None	$\max(0.0015625 \text{ from A}, 0.01875 \text{ from B}) = 0.01875$	0.01875

The audit executed this example against DeviceABM and obtained these exact values. If all four illustrative nodes have sensor criticality 0.5, the defended blast radius is $0.5 * (0.8 + 0.6 + 0.15 + 0.01875) = 0.784375$.

9. Multiple propagation sources

Contributions do not sum, average, or form a probabilistic union. Each source has its own traversal, and the shared result uses:

```
if pushed > new_propagated.get(neighbor, 0.0):
    new_propagated[neighbor] = pushed
```

(simulation/abm.py:146-166). Thus:

P_i = max over source traversals of the retained candidate for i

Example:

A (direct 0.8) ---- C ---- B (direct 0.6)

A sends $0.8 * 0.25 = 0.20$ to C. B sends $0.6 * 0.25 = 0.15$ to C. C receives $\max(0.20, 0.15) = 0.20$, not 0.35. Because `all-neighbor` traversal is bidirectional, A also receives 0.15 from B over two edges only if C is the

intermediate; the exact two-edge value is $0.6 * 0.03125 = 0.01875$, while B receives 0.025 from A.

This maximum policy prevents accumulation across independent sources. It is a design choice, not an inference about correlated or independent attack probabilities.

10. Cycle handling

For every source, visited begins with the source and is shared across that source's entire traversal (simulation/abm.py:146-149). Therefore:

- the source cannot be revisited through a cycle;
- a node is expanded at most once per source;
- cycles cannot run indefinitely;
- max_hops provides a second independent termination bound;
- risk does not repeatedly multiply around a loop.

However, the first discovered path from one source wins. Once a neighbor is visited, a later path from the same source is ignored before its candidate is calculated (simulation/abm.py:154-166). Under default homogeneous coefficients below 1, breadth-first discovery normally selects a shortest path, and shorter paths are stronger. With custom amplifying or otherwise unusual parameters, a later, longer path could be stronger but is still ignored. Neighbor insertion order could then affect results.

The topology's sensor/camera/plug rule sets are Python sets (simulation/topology.py:22-38,62-64,77-82), so insertion order is not a robust cross-process ordering contract. Under current decreasing defaults, equal-distance paths produce equal numeric values, limiting this issue. Parameters are not range validated or output-clipped in propagate(); custom $w > 1$ or $d > 1$ can exceed the nominal [0,1] risk scale (simulation/abm.py:131-166).

test_cycles_cannot_amplify_without_bound constructs a cycle, repeats propagation, and checks default-case values remain ≤ 1 (tests/unit/runtime/test_topology_abm_srep.py:140-155). It does not verify exact path selection or custom-parameter order independence.

11. Protected assets, attackers, and cloud nodes

ABM construction uses:

```
is_protected_asset = role not in ("attacker", "cloud")
is_attacker = role == "attacker"
```

(simulation/abm.py:72-91). Propagation then imposes two filters:

- only protected nodes can be sources (simulation/abm.py:135-144);
- every neighbor must have an ABM state and be protected, or traversal skips it

(simulation/abm.py:154-159).

Consequences:

Node class	Can hold direct network risk?	Can originate propagation?	Can receive propagation?	Can relay propagation?
protected asset	yes	yes, if direct > 0	yes	yes
attacker	yes, if gateway accepts known identity	no	no	no
cloud	yes, if gateway accepts known identity	no	no	no

The gateway resolves against all ABM states, not only protected states (agents/finding_gateway.py:96-109). A known attacker/cloud network finding can therefore set direct and systemic state, but it cannot propagate. Behavior findings for non-sensors fail apply_behavior_evidence() (simulation/abm.py:113-118).

Risk cannot cross a non-protected intermediate node. The attacker C2 component and cloud edge exist structurally (simulation/topology.py:68-74) but are removed in effect from the propagation traversal. This is a significant simulation assumption: attackers provide recorded attack traffic and may have direct detector scores, but the propagation model excludes them as risk origins and bridges.

Defended blast radius also excludes attackers and every non-protected node (simulation/abm.py:181-189).

12. Systemic risk

Systemic risk is calculated in DeviceABM.propagate() after propagated risk is replaced (simulation/abm.py:168-179):

```
if direct_risk is None:
    systemic_risk = propagated_risk
else:
    systemic_risk = max(direct_risk, propagated_risk)
```

Equivalently, when direct values are present:

$S_i = \max(N_i, B_i, P_i)$, ignoring N_i or B_i when it is None

Cases:

Case	Direct	Propagated	Systemic	Interpretation
A: high direct, low propagated	0.8	0.2	0.8	direct dominates
B: low direct, high propagated	0.2	0.8	0.8	topology result dominates
C: no direct evidence	None	0.3	0.3	propagated evidence only
D: no direct or propagated evidence	None	0.0	0.0	simulation default, not a dataset observation of safety

Systemic risk is best described as a deterministic max-fused simulation risk score. It is not a probability of node compromise, a probability of whole-system compromise, a Bayesian posterior, or accumulated probability across paths. No code models dependence between network and behavior scores or calibrates them onto a common probabilistic scale before taking their maximum.

The workflow RiskAnalyst does not recalculate this value. It reads and passes through authoritative ABM state (agentic_workflow/risk_analyst.py:56-77,121-142). Its test deliberately supplies an inconsistent systemic value and verifies that it is preserved (tests/unit/agentic_workflow/test_risk_analyst.py:43-49).

13. Compromised flag versus systemic risk

DeviceState.compromised begins False (simulation/abm.py:33-55). The only assignment to True is:

```
if state.is_protected_asset and finding.attack_probability >= 0.5:
    state.compromised = True
```

(simulation/abm.py:103-110). Therefore:

- only accepted network evidence can set compromise;
- the threshold is exactly 0.5;
- behavior risk cannot set it;
- propagated risk cannot set it;
- systemic risk cannot set it;
- attackers/cloud cannot be marked compromised because they are not protected;
- no implementation resets it to false.

A node can consequently have high propagated or systemic risk and still have compromised=False. Conversely, once a protected node receives network risk ≥ 0.5 , later low network scores can replace network_risk but cannot clear compromise. This flag is a sticky project rule, not ground truth from DataSense and not the same concept as systemic risk.

14. Defended blast radius

DeviceABM.defended_blast_radius() implements (simulation/abm.py:181-189):

```
BR = round(sum over protected nodes i of  $S_i * C(\text{role}_i)$ , 6)
```

The function explicitly skips attackers and all non-protected nodes. Criticality is looked up by role, falling back to default_criticality (simulation/abm.py:183-188). Defaults are in config.py:39-57:

Role key	Criticality
mqtt-broker	1.0
edge1	0.9
router	0.9
switch	0.8
ap	0.8
sensor	0.5
camera	0.6
smart-plug	0.4
attacker	0.0
cloud	0.7
default	0.5

These are simulation-defined constants, not DataSense measurements (config.py:24-27,39-57, srep/device_srep.py:11-13,91-98).

Worked example: a sensor at systemic 0.8 contributes $0.8 * 0.5 = 0.4$; a router at systemic 0.2 contributes $0.2 * 0.9 = 0.18$; defended blast radius is 0.58. It is an unnormalized weighted aggregate simulation score. It is neither a count of compromised devices nor a probability, and it can exceed 1.

Configuration mismatch: the inventory gives device edge1 role edge (data/raw/datasense/docs/site/devices.csv:4), while the criticality map key is edge1 (config.py:44-47). Since lookup uses role, edge1 receives default 0.5, not configured 0.9. This is a verified implementation/configuration mismatch and a possible bug.

SREP does not recalculate propagation. It projects all ABM states, calculates each protected node's displayed contribution, sorts top protected nodes by systemic risk, and reports abm.defended_blast_radius() (srep/device_srep.py:52-108). Communication graph data contributes only displayed node/edge counts, not risk

(srep/device_srep.py:82-85). SREP explicitly remains DEVICE_ONLY; a trust graph is rejected (srep/device_srep.py:37-50).

15. Replay timing

Base ReplayRunner path

Without a workflow callback, each selected window runs in this order:

```
select lowest available window ID
-> begin communication delta window
-> drain all network, behavior, and communication rows for that ID
-> apply communication rows to separate communication graph
-> infer all eligible network findings
-> gateway-submit each network finding
-> infer behavior findings
-> gateway-submit each behavior finding
-> set ABM current window
-> propagate once
-> record_step once
-> publish WINDOW_COMPLETED
```

See simulation/replay.py:218-245,287-369. Input streams are externally sorted by window_id and merged by the minimum head (simulation/replay.py:144-190,218-233). Thus direct evidence from both modalities is applied before propagation in the base path. Propagation is once per processed window, not once per finding or once per replay. record_step() follows propagation once per processed window.

No-observation and communication-only windows still call propagation and retain old direct state (simulation/replay.py:287-360). Missing window IDs are skipped because the runner processes the union of IDs present in its input heads.

Current FastAPI five-agent path

The backend controller attaches WorkflowService.execute_window() as the runner's callback (backend/app/services/replay_controller.py:440-461). That changes the normal timing:

1. Workflow sets abm.current_window_id at entry (backend/app/services/workflow_service.py:285-305).
2. Network and behavior specialists are selected through readiness/orchestration. Each submits findings through the same gateway (backend/app/services/workflow_service.py:334-360,382-429).
3. When both detector roles are complete, workflow calls abm.propagate() and abm.record_step() once (backend/app/services/workflow_service.py:398-405,421-429).
4. The later risk-propagation analyst stage calls abm.propagate() again, then reads the resulting ABM state; it does not call record_step() (backend/app/services/workflow_service.py:481-497).
5. ReplayRunner only performs a fallback propagation/record if the current window differs from the target. Workflow set it at entry, so this fallback normally does not run (simulation/replay.py:249-285).

Therefore a successful current backend window normally executes propagation twice but records one ABM step. With unchanged direct evidence, the second call recomputes the same deterministic values from scratch. Workflow failures can alter this count; several exceptions around propagation are swallowed (backend/app/services/workflow_service.py:398-405,421-429,481-485).

Normal sorted replay does not accept late findings after a window snapshot. However, the gateway/ABM itself has no check that a finding's window_id is newer than the last channel update; an externally or incorrectly ordered older finding could overwrite direct risk and timestamp (simulation/abm.py:103-123).

16. FindingGateway boundary

Models return NetworkFinding or BehaviorFinding; they do not receive an ABM reference (pipeline/network_detector.py:142-209, pipeline/behavior_profiler.py:281-344). Finding dataclasses validate score bounds, timestamp shape, class/profile vocabulary, and whitelisted provenance at construction (pipeline/findings.py:23-74,90-152).

FindingGateway.submit() then:

1. accepts only registered finding types;
2. revalidates a timestamp is present and parseable with an explicit offset;
3. checks concrete finding class;
4. resolves entity_id against ABM state;
5. routes network and behavior findings to separate mutation methods;

6. appends accepted metadata to a bounded log;
7. invokes subscribers only after acceptance.

See `agents/finding_gateway.py:40-123`. Statistics and recent rejections are bounded in `GatewayStats` (`agents/finding_gateway.py:40-48`). Unknown entities and invalid types are rejected without state mutation. Rejected findings are never passed to `apply_*_evidence()`, so they cannot become propagation sources.

Behavior support is enforced in `DeviceABM.apply_behavior_evidence()`, not in a pre-check inside the gateway (`simulation/abm.py:113-119`). That exception is not caught by `FindingGateway.submit()`. Provenance is deeply constrained when the finding is constructed (`pipeline/findings.py:23-74`), but the gateway does not re-run provenance validation; because a frozen dataclass can still contain a mutable dict, post-construction provenance mutation is a caveat.

Subscribers receive accepted findings after ABM mutation (`agents/finding_gateway.py:111-123`). The production adapter uses this to expose accepted findings to the Blackboard without giving it a second ABM mutation path (`backend/app/adapters/stage2_replay_adapter.py:84-93`).

17. Original dataset versus simulation

Concept	Original dataset	Extracted	ML-derived	Simulation-defined
packet observations	YES: PCAP bytes/timestamps/lengths	decoded into FrameView	no	no
MQTT/telemetry observations	YES: NDJSON fields	compact MqttEvent	no	no
network features	no	YES: 57 project aggregate fields	model consumes them	no
behavior features	no	YES: common/profile-specific aggregates	profiler consumes them	profile grouping is project-defined
network attack probability	no	no	YES: raw RF class-1 score	copied into ABM state
behavioral deviation score	no	no	YES: Isolation Forest transform or rules	rules/score mappings are project-designed
topology	inventory nodes and documented testbed evidence only	no	no	PARTLY: executable edge set is hard-coded/documented/strongly inferred
propagation weight	no	no	no	YES: 0.5
hop decay	no	no	no	YES: 0.5
max hops	no	no	no	YES: 3
propagated risk	no	no	no	YES: DeviceABM.propagate()
systemic risk	no	no	no	YES: max fusion in ABM
role criticality	roles exist in inventory; numeric criticality does not	no	no	YES: configuration map
defended blast radius	no	no	no	YES: weighted systemic sum

Evidence: raw structures are executable in `datasets/datasense/pcap_reader.py:41-47`, `datasets/datasense/frame_decoder.py:55-83`, and `datasets/datasense/ndjson_reader.py:25-40`; the prior repository raw audit lists the verified PCAP fields and MQTT schema (`docs/datasense_raw_audit.md:95-145,147-197`). Feature families are defined in `datasets/datasense/network_features.py:48-118` and `datasets/datasense/behavior_features.py:43-98`. Model and simulation origins are in `pipeline/network_detector.py:117-176`, `pipeline/behavior_profiler.py:281-432`, and `simulation/abm.py:103-189`.

The repository methodology independently classifies raw observations as dataset-grounded, features/predictions/findings as project-derived, and propagation constants as simulation-defined (`docs/datasense_raw_pipeline_methodology.md:582-616`).

18. Parameter provenance

Propagation weight

```
name: propagation_weight
default: 0.5
defined in: config.py:39-43
used in: simulation/abm.py:131,160
dataset-derived?: no
learned?: no
simulation-defined?: yes
configurable?: programmatically through DeviceABM(srep_params=...)
documented as simulation-defined?: yes, config.py:24-27 and srep/device_srep.py:11-13
```

Hop decay

```
name: hop_decay
default: 0.5
defined in: config.py:39-43
used in: simulation/abm.py:132,160
dataset-derived?: no
learned?: no
simulation-defined?: yes
configurable?: programmatically through DeviceABM(srep_params=...)
documented as simulation-defined?: yes
```

Maximum hops

```

name: max_hops
default: 3
defined in: config.py:39-43
used in: simulation/abm.py:133,152
dataset-derived?: no
learned?: no
simulation-defined?: yes
configurable?: programmatically through DeviceABM(srep_params=...)
documented as simulation-defined?: yes

```

Criticality map and default

```

name: criticality / default_criticality
default: role map shown in section 14; fallback 0.5
defined in: config.py:44-57
used in: simulation/abm.py:181-189 and srep/device_srep.py:52-75
dataset-derived?: role labels are dataset inventory; numeric weights are not
learned?: no
simulation-defined?: yes
configurable?: programmatically through ABM/SREP params
documented as simulation-defined?: yes

```

Network class and compromise threshold

```

name: attack/compromise threshold
default: 0.5
defined in: pipeline/network_detector.py:137 and simulation/abm.py:109
used in: predicted_class and sticky protected-node compromise
dataset-derived?: no
learned?: no
simulation-defined?: yes, hard-coded policy
configurable?: not through configuration in current implementation
documented as simulation-defined?: not individually in config

```

The same literal is duplicated in model classification and ABM compromise logic; ABM uses the numeric probability, not predicted_class.

Behavioral thresholds

```

continuous tau:
  default/source: approximately 99th percentile of later calibration scores
  defined/used: pipeline/behavior_profiler.py:177-180,198,330,404
  learned/calibrated?: derived per device from benign calibration rows

sparse tau:
  default/source: approximately 99th percentile of calibration rule scores
  defined/used: pipeline/behavior_profiler.py:258-277,430
  learned/calibrated?: threshold is data-derived; rule increments are fixed

absence_tau_windows:
  default/source: max(3, int(p90(training active-window gaps)) + 1)
  defined/used: pipeline/behavior_profiler.py:232-235,266-272,365-370
  learned/calibrated?: data-derived from training gaps, with simulation floor 3

fixed behavior constants:
  sparse increments: 0.40, 0.35, up to 0.25, 0.15
  absence ramp: 0.5 + 0.1 * over
  degenerate guards: minimum 0.85 / 0.8 and 5 percent drift
  source: pipeline/behavior_profiler.py:237-256,369-380,393-403,409-432
  learned?: no
  simulation/project-defined?: yes

```

Fusion and cutoff

```

name: systemic fusion
default: max(present network, present behavior, propagated)
defined/used: simulation/abm.py:168-179
learned?: no
configurable?: no

name: propagation cutoff
default: discard candidate <= 1e-9
defined/used: simulation/abm.py:160-162
learned?: no
configurable?: no

```

Production does not expose these scientific parameters as backend environment settings. build_runtime() constructs DeviceABM without an override (backend/app/adapters/stage2_replay_adapter.py:73-87).

19. Tests and guarantees

Focused verification performed during this audit:

```
python -m pytest tests/unit/runtime/test_topology_abm_srep.py \
    tests/unit/runtime/test_findings_gateway.py \
    tests/unit/modeling/test_behavior_profiler.py \
    tests/unit/features/test_observation_masks.py -q
Result: 26 passed

python -m pytest tests/integration/backend/api/test_model_instance_isolation.py -q
Result: 3 passed

| Invariant | Test | Status |
| direct risk channels kept separate |
tests/unit/runtime/test_findings_gateway.py:test_network_finding_updates_only_network_evidence and
test_behavior_finding_updates_only_behavior_evidence, lines 76-93 | Verified by test |
| direct and propagated remain separate |
tests/unit/runtime/test_topology_abm_srep.py:test_direct_vs_propagated_separate_and_decaying, lines 126-137 | Verified, but only
broad decay bounds |
| behavior missing != zero | tests/unit/modeling/test_behavior_profiler.py:test_unsupported_missing_behavior_is_not_risk_zero,
lines 86-94; contract round-trip at tests/integration/backend/api/test_contracts.py:59-71 | Verified for initial/contract state
|
| unobserved network row cannot infer risk |
tests/regression/pipeline/test_corrective_pass.py:test_unobserved_rows_never_generate_findings_or_risk, lines 148-167 | Verified
by test |
| sparse absence requires active context |
tests/regression/pipeline/test_corrective_pass.py:test_sparse_absence_on_dense_rows_with_active_context, lines 395-454 |
Verified by direct profiler test |
| propagation deterministic | repeated propagation occurs in cycle test, tests/unit/runtime/test_topology_abm_srep.py:140-155 |
Not directly verified for order/path independence |
| exact hop formula | none found | Not directly verified |
| hard hop cap respected | implementation at simulation/abm.py:149-166; cycle test changes cap | Not directly verified by
numeric oracle |
| cycles bounded | tests/unit/runtime/test_topology_abm_srep.py:test_cycles_cannot_amplify_without_bound, lines 140-155 |
Verified under tested/default coefficients |
| max across sources, not sum | none found | Not directly verified |
| rejected finding does not mutate known state |
tests/integration/backend/workflow/test_workflow_integration.py:test_finding_gateway_remains_authoritative, lines 237-261 |
Verified for unknown entity |
| rejected finding does not reach Blackboard |
tests/integration/backend/blackboard/test_blackboard_pipeline_integration.py:182-217 | Verified for unknown entity |
| accepted finding applies once | tests/integration/backend/blackboard/test_blackboard_pipeline_integration.py:219-265 |
Verified with counting stub |
| replay model-state isolation | tests/integration/backend/api/test_model_instance_isolation.py:19-78 | Verified for
detector/profiler identities and profiler sentinel state |
| full ABM/gateway/graph isolation | none found | Not directly verified as object identities |
| attacker excluded from blast contribution | tests/unit/runtime/test_topology_abm_srep.py:171-198 | Verified for zero SREP
contribution; blast-radius assertion is weak |
| systemic uses exact max fusion | implementation at simulation/abm.py:168-179; analyst pass-through test does not calculate it
| Not directly verified by an independent numeric test |
| topology metadata/freeze | tests/unit/runtime/test_topology_abm_srep.py:87-101 | Verified for sampled edges and frozen
structure |
| communication graph does not mutate topology | tests/unit/runtime/test_topology_abm_srep.py:112-123 | Verified by test |
| SREP rejects trust graph | tests/unit/runtime/test_topology_abm_srep.py:224-231 | Verified by test |
| chronological windows | tests/integration/backend/api/test_event_chronology.py:6-30; external sorter at
tests/unit/storage/test_window_sort_fanin.py:16-31,68-76 | Verified for tested replay and sorter |
| direct-raw/store scientific equivalence | tests/regression/pipeline/test_replay_equivalence.py:124-187 | Regression
equivalence, not formula correctness |
```

The focused tests passed in this checkout. Artifact-dependent regression tests were inspected but not included in the focused run above. Equivalence tests demonstrate that two input modes use the same science; they do not independently validate the science's formula.

20. Potential implementation caveats

```
| Classification | Caveat | Evidence and impact |
| DESIGN CHOICE | Propagation uses maximum, not additive accumulation. | simulation/abm.py:164-165. Independent sources cannot
accumulate exposure. |
| IMPORTANT ASSUMPTION | A directed topology is traversed through predecessors and successors. | simulation/topology.py:41-84,
simulation/abm.py:154; verified with installed NetworkX 3.6.1. Displayed arrow direction is not a propagation constraint. |
| IMPORTANT ASSUMPTION | The recursive formula produces triangular hop-decay exponents. | simulation/abm.py:149-166. Third-hop
value is  $D \cdot w^3 \cdot d^6$ , not  $D \cdot w \cdot d^3$ . |
| POSSIBLE BUG | First-discovery visited can suppress a stronger alternate path under custom amplifying parameters. |
simulation/abm.py:154-166. Defaults are monotonically decreasing, which mitigates it. |
| DESIGN CHOICE | Propagated risk is recalculated from scratch each call. | new_propagated={} and replacement at
simulation/abm.py:146,168-169. It does not accumulate across windows. |
| IMPORTANT ASSUMPTION | Direct risk and observation flags persist across no-observation windows. | No reset exists around
simulation/abm.py:103-123; replay still propagates old state at simulation/replay.py:287-360. Frontend state is latched replay
state, not current-window evidence only. |
| IMPORTANT ASSUMPTION | Later accepted findings overwrite earlier direct values without stale-window checks. |
```

simulation/abm.py:103-123. An older finding could overwrite a newer score if admitted out of order. |
DESIGN CHOICE	Compromise is sticky, protected-only, and network-only.	simulation/abm.py:103-110. High behavior/propagated/systemic risk does not set it.
RESEARCH LIMITATION	Network and behavior scores are treated as numerically comparable by max.	simulation/abm.py:139-142,170-179. One is an uncalibrated RF class score; the other is an anomaly/rule score.
RESEARCH LIMITATION	Propagation weight, decay, hop cap, and criticality are not empirically learned or calibrated from DataSense.	config.py:24-57, srep/device_srep.py:11-13,91-98.
RESEARCH LIMITATION	Systemic risk has no probabilistic interpretation or uncertainty propagation.	Max fusion and max-source aggregation at simulation/abm.py:146-179.
IMPORTANT ASSUMPTION	Non-protected attackers/cloud cannot originate, receive, or relay propagated risk.	simulation/abm.py:135-159. Risk cannot cross them as intermediate nodes.
POSSIBLE BUG	edgel criticality key does not match its inventory role edge.	config.py:44-57, data/raw/datasense/docs/site/devices.csv:4, lookup simulation/abm.py:186-188. Actual criticality is fallback 0.5, not 0.9.
DOCUMENTATION GAP	"Direct evidence never overwritten" can be misread.	Propagation does not overwrite direct channels, but later findings do at simulation/abm.py:103-123; compare methodology docs/datasense_raw_pipeline_methodology.md:688-693.
DOCUMENTATION GAP	ABM *_observed flags are historical/accepted-evidence latches.	They are set true and never reset (simulation/abm.py:103-123). Sparse inferred absence also sets behavior observed.
POSSIBLE BUG	Degenerate constant-stream guard is neutralized by extraction forcing true.	datasets/datasense/behavior_features.py:489-493 versus pipeline/behavior_profiler.py:393-398.
POSSIBLE BUG	Custom propagation parameters are not range-validated or clipped.	simulation/abm.py:131-166; output can exceed [0,1], while workflow contracts expect [0,1].
POSSIBLE BUG	Custom SREP params can disagree with ABM blast-radius params.	SREP uses self.params for node contributions but calls abm.defended_blast_radius() (srep/device_srep.py:31-35,52-73,99). Production uses the same params, so the normal path is consistent.
POSSIBLE BUG	Behavior support is role-based in ABM but named-profile-based in inventory/profiler.	simulation/abm.py:72-91 versus datasets/datasense/devices.py:113-123. Known inventory sensors are covered; custom unknown sensors could disagree.
IMPORTANT ASSUMPTION	Current workflow can run propagation twice per successful window.	backend/app/services/workflow_service.py:398-429,481-497. Values are recomputed, not doubled.
POSSIBLE BUG	Workflow propagation exceptions are swallowed.	backend/app/services/workflow_service.py:398-405,421-429,481-485 and callback wrapper backend/app/services/replay_controller.py:442-459. Failed risk calculation may be underreported.
DESIGN CHOICE	Frontend clamps only bar width, not displayed numeric value.	frontend/src/components/devices/DeviceStateTable.tsx:9-11,110-121. This preserves backend value but can visually conceal out-of-range custom results.
RESEARCH LIMITATION	Current saved network artifact is a smoke model, not a research-grade result.	models/saved_models/split_manifest_smoke.json:26-52; validation recall 0 and test has zero positives.

None of these caveats demonstrates that DataSense itself contains risk propagation. They concern the project's simulation semantics and robustness.

21. Required claim verification

Claim 1

CLAIM: "The original DataSense corpus contains no propagated-risk field." VERDICT: VERIFIED within repository evidence EVIDENCE: Raw executable records contain packet fields or MQTT fields only (datasets/datasense/pcap_reader.py:41-47, datasets/datasense/ndjson_reader.py:25-40). The repository's prior raw audit lists verified PCAP and MQTT schemas without risk fields (docs/datasense_raw_audit.md:95-145,147-197). REASONING: propagated_risk first appears as initialized simulation state and is assigned by ABM propagation (simulation/abm.py:48-49,126-169). The raw corpus was not present to rescan independently during this audit.

Claim 2

CLAIM: "Risk propagation is synthesized by DeviceABM.propagate()." VERDICT: VERIFIED EVIDENCE: simulation/abm.py:126-169. REASONING: The method selects direct sources, traverses topology, calculates decayed candidates, takes maxima, and assigns every propagated_risk.

Claim 3

CLAIM: "The propagation parameters are simulation-defined rather than learned from DataSense." VERDICT: VERIFIED EVIDENCE: config.py:24-27,39-57; srep/device_srep.py:11-13,91-98. REASONING: Defaults are constants, no fit path estimates them, and SREP explicitly disclaims DataSense measurement.

Claim 4

CLAIM: "The network model produces direct evidence, not propagated risk." VERDICT: VERIFIED EVIDENCE: pipeline/network_detector.py:117-176; pipeline/findings.py:90-118; simulation/abm.py:103-111. REASONING: Model output is NetworkFinding.attack_probability; ABM copies it to the direct network channel. The model has no topology input or propagated-risk output.

Claim 5

CLAIM: "The behavioral profiler produces direct behavioral deviation evidence, not propagated risk." VERDICT: VERIFIED EVIDENCE: pipeline/behavior_profiler.py:281-344; pipeline/findings.py:121-155; simulation/abm.py:113-123. REASONING: Output is BehaviorFinding.deviation_score; it is copied to the behavior channel. The profiler has no topology propagation output.

Claim 6

CLAIM: "Systemic risk is created by the ABM, not read from the dataset." VERDICT: VERIFIED EVIDENCE: simulation/abm.py:168-179; raw schemas cited under claim 1. REASONING: Systemic risk is assigned only after direct/propagated max fusion.

Claim 7

CLAIM: "Missing behavioral evidence is ignored rather than averaged as zero." VERDICT: VERIFIED EVIDENCE: Candidate construction excludes None before max (simulation/abm.py:170-179); initial null preservation is tested at tests/unit/modeling/test_behavior_profiler.py:86-94 and transport distinction at tests/integration/backend/api/test_contracts.py:59-71. REASONING: At source eligibility, missing and zero both fail to create a positive source, but final direct fusion preserves missingness and performs no average.

Claim 8

CLAIM: "Propagation is max-based rather than additive." VERDICT: VERIFIED from implementation; not directly test-oracled EVIDENCE: simulation/abm.py:164-165. REASONING: The result changes only when a new candidate exceeds the stored value; there is no addition.

Claim 9

CLAIM: "Propagation has a hard hop cap." VERDICT: VERIFIED from implementation EVIDENCE: config.py:41-43; simulation/abm.py:149-153,166. REASONING: Nodes at hops $\geq$ max_hops are not expanded. No independent exact-hop numeric unit test was found.

Claim 10

CLAIM: "Attackers are excluded from defended blast radius." VERDICT: VERIFIED EVIDENCE: simulation/abm.py:181-189; test tests/unit/runtime/test_topology_abm_srep.py:171-198. REASONING: Radius skips attackers and every non-protected state; SREP gives the tested attacker zero defended contribution.

Claim 11

CLAIM: "A high systemic risk does not necessarily set compromised=True." VERDICT: VERIFIED from implementation EVIDENCE: The only compromise assignment is network-only at simulation/abm.py:103-110; systemic assignment has no compromise side effect at simulation/abm.py:168-179. REASONING: A node can receive high propagated risk while never receiving a network finding ≥ 0.5 . No direct automated test specifically asserts this contrast.

Claim 12

CLAIM: "The FindingGateway is an admission/validation boundary before mutable ABM state." VERDICT: VERIFIED EVIDENCE: pipeline/findings.py:1-12; agents/finding_gateway.py:51-123; gateway tests tests/unit/runtime/test_findings_gateway.py:76-180. REASONING: Models emit values; gateway validates type/timestamp/entity and only then invokes one ABM mutation method. Rejected unknown findings do not mutate state or notify subscribers.

What this repository's risk propagation actually means

The defensible interpretation is:

> Recorded DataSense packet and telemetry observations are converted into project > features. A network classifier and per-sensor behavioral profiler produce direct > scores. The simulation then projects the maximum direct score through a static, > project-constructed topology using fixed decay constants and max fusion.

The repository does not establish:

> DataSense measured an attack physically propagating from one device to another, > or measured the probability/weight/decay of that propagation.

Dataset-derived

- PCAP packet timestamps, lengths, and frame bytes.
- Decodable MAC/IP/protocol/port/flag/header observations.
- MQTT timestamps, values, topics, message metadata, MAC/IP identity.
- Inventory names, roles, types, addresses, and documented testbed evidence.

Evidence: datasets/datasense/pcap_reader.py:41-47, datasets/datasense/frame_decoder.py:55-108, datasets/datasense/ndjson_reader.py:25-90, data/raw/datasense/docs/site/devices.csv:1-46.

Extracted and model-derived

- Project-defined device/window feature rows and observation masks.
- Random Forest class-1 network score, class, and confidence.
- Per-sensor behavioral anomaly/rule score, explanation, and confidence.
- Accepted direct network_risk and behavior_risk values.

Evidence: datasets/datasense/network_features.py:48-118,426-545, datasets/datasense/behavior_features.py:43-98,397-526, pipeline/network_detector.py:117-176, pipeline/behavior_profiler.py:281-432, simulation/abm.py:103-123.

Simulation-defined

- Static propagation topology as executable edge rules.
- Protected/non-protected propagation filtering.
- Propagation weight 0.5, hop decay 0.5, maximum three hops.
- Maximum across sources and maximum direct/propagated fusion.
- Sticky network threshold compromise rule.
- Role criticalities and defended blast radius.
- DEVICE_ONLY SREP projection.

Evidence: simulation/topology.py:41-84, simulation/abm.py:126-189, config.py:24-57, srep/device_srep.py:37-108.

Not established by this repository

- Empirical DataSense propagation coefficients or causal attack spread.
- Calibrated probability meaning for network score, behavior score, propagated score, systemic score, or blast radius.
- Bayesian or additive multi-path risk.
- Agent-trust-graph fusion or full dual-graph systemic risk.
- Research-grade attack generalization by the current smoke network model.
- A validated claim that directed topology arrows constrain propagation direction.

Compact audit reference

Value	Exact repository meaning	Formula/source	Important limitation
network_risk	latest accepted RF class-1 network score	$N_i = \text{finding.attack_probability}$	raw, uncalibrated classifier score; latched
pipeline/network_detector.py:117-176, simulation/abm.py:103-111	behavior_risk	latest accepted per-sensor deviation score	$B_i = \text{finding.deviation_score}$
pipeline/behavior_profiler.py:281-432, simulation/abm.py:113-123	direct risk	local max of present direct channels	$D_i = \max(\text{present } N_i, B_i)$; simulation/abm.py:170-175 not stored in DeviceState; score scales assumed comparable
simulation/abm.py:126-169	propagated_risk	strongest simulated topology contribution	$P_i = \max_s D_s * w^k * d^{(k(k+1)/2)}$ on first visited path; non-additive; effectively bidirectional; protected-only; constants not learned
simulation/abm.py:168-179	systemic_risk	deterministic max-fused device risk	$S_i = \max(D_i, P_i)$, or P_i if direct missing; not probability of compromise/system compromise
simulation/abm.py:103-110	compromised	sticky protected-device network alarm flag	true after accepted attack_probability ≥ 0.5 ; behavior/propagation/systemic cannot set or reset it
simulation/abm.py:181-189	defended blast radius	criticality-weighted protected systemic sum	$\text{round}(\sum S_i * C(\text{role}_i), 6)$; unnormalized simulation aggregate, can exceed 1

Final conclusion

Risk propagation is synthesized by the simulation. The original DataSense data provides observations and inventory/testbed evidence; the network and behavior models provide direct scores. DeviceABM.propagate() alone creates propagated and systemic state using a static topology and simulation-defined coefficients. SREP, the backend device-state endpoint, and the frontend display those results; they do not supply an independent propagation model (simulation/abm.py:126-189, srep/device_srep.py:52-108, backend/app/adapters/stage2_replay_adapter.py:195-228, frontend/src/components/devices/DeviceStateTable.tsx:69-75).